



Player & UI

YOU'LL SHIP

An intro screen, a main menu and an in-game HUD.

- 4.1 Build a studio-grade intro screen
- 4.2 ScreenGui, Frames and TextLabels
- 4.3 Scaling for desktop, mobile and console
- 4.4 Animation with TweenService
- 4.5 Main menu and HUD

UI is the first thing players judge. A clean intro, a readable HUD and a menu that works on a phone make a game feel trustworthy before anyone has played a single round. In this module you'll build all three for Coin Rush.

1 Build a studio-grade intro screen

This is the free preview lesson, in short. (The full step-by-step version with explanations is in [free-lesson-intro-screen.md](#).)

Put a **LocalScript** named `IntroSplash` in **ReplicatedFirst**, which is the first thing that runs when a player joins:

```

local ReplicatedFirst = game:GetService("ReplicatedFirst")
local Players = game:GetService("Players")
local TweenService = game:GetService("TweenService")

ReplicatedFirst:RemoveDefaultLoadingScreen()

local playerGui = Players.LocalPlayer:WaitForChild("PlayerGui")

local gui = Instance.new("ScreenGui")
gui.IgnoreGuiInset = true
gui.DisplayOrder = 1000
gui.ResetOnSpawn = false

local bg = Instance.new("Frame")
bg.Size = UDim2.fromScale(1, 1)
bg.BackgroundColor3 = Color3.new(0, 0, 0)
bg.BorderSizePixel = 0
bg.Parent = gui

local logo = Instance.new("ImageLabel")
logo.Image = "rbxassetid://YOUR_ASSET_ID"
logo.BackgroundTransparency = 1
logo.AnchorPoint = Vector2.new(0.5, 0.5)
logo.Position = UDim2.fromScale(0.5, 0.5)
logo.Size = UDim2.fromScale(0.4, 0.4)
logo.ImageTransparency = 1
logo.Parent = bg
Instance.new("UIAspectRatioConstraint", logo).AspectRatio = 1
local scale = Instance.new("UIScale", logo)
scale.Scale = 0.9

gui.Parent = playerGui

TweenService:Create(scale, TweenInfo.new(0.25, Enum.EasingStyle.Back), { Scale = 1 }):Play()
local fadeIn = TweenService:Create(logo, TweenInfo.new(0.25), { ImageTransparency = 0 })
fadeIn:Play()
fadeIn.Completed:Wait()
task.wait(1.75)

local out = TweenInfo.new(0.4)
TweenService:Create(logo, out, { ImageTransparency = 1 }):Play()
local bgOut = TweenService:Create(bg, out, { BackgroundTransparency = 1 })

```

```

bgOut:Play()
bgOut.Completed:Wait()
gui:Destroy()

```



ScreenGui, Frames and TextLabels

All 2D interface lives inside a **ScreenGui**. You design UI in **StarterGui**; when a player spawns, Roblox copies it into their **PlayerGui**.

The building blocks:

Class	Use
Frame	A rectangle, used as a panel or container.
TextLabel	Text that players read.
TextButton / ImageButton	Things players click or tap.
ImageLabel	Pictures and icons.
UICorner	Rounded corners.
UIStroke	Outlines, for text or frames.
UIListLayout / UIGridLayout	Automatic arrangement of children.
UIPadding	Inner spacing.

Build the HUD layout in StarterGui:

```

StarterGui
├─ HUD (ScreenGui)
│   ├── CoinPanel (Frame, top-left)
│   │   ├── UICorner
│   │   ├── Icon (ImageLabel)
│   │   └─ CoinText (TextLabel) "0"
│   └─ TimerPanel (Frame, top-center)
│       ├── UICorner
│       └─ TimerText (TextLabel) "Intermission"

```

Text tips: use a bold, readable font (**FontFace** → Gotham/Montserrat/Fredoka One style), turn on **TextScaled** with a **UITextSizeConstraint** (MaxTextSize ~36), and add a **UIStroke** so text stays readable on bright backgrounds.

🔧.3 Scaling for desktop, mobile and console

More than half of Roblox players are on phones. UI that looks perfect on your monitor can be unusable on a phone.

Rule 1: use Scale, not Offset. A `UDim2` has four numbers: `(xScale, xOffset, yScale, yOffset)`. Scale is a fraction of the screen; Offset is pixels.

```
UDim2.new(0.2, 0, 0.08, 0)    -- 20% of width, 8% of height: works everywhere
UDim2.new(0, 300, 0, 60)     -- 300x60 px: huge on phones, tiny on 4K
```

Rule 2: lock shapes with `UIAspectRatioConstraint`. A coin icon should stay round on every screen.

Rule 3: position with `AnchorPoint`. `AnchorPoint = (0.5, 0)` plus `Position = (0.5, 0, 0.02, 0)` keeps the timer centered at the top on any screen.

Rule 4: respect the safe area. Phones have notches and Roblox has its top-bar buttons. Keep important UI a little away from the edges, or check `ScreenGui.ScreenInsets`.

Test it: in the **Test** tab open the **Device emulator** and switch between phone, tablet and console sizes. Fix anything that overlaps or gets too small.

🔧.4 Animation with TweenService

`TweenService` smoothly animates any property from its current value to a target.

```
local TweenService = game:GetService("TweenService")

local info = TweenInfo.new(
    0.3, -- duration (seconds)
    Enum.EasingStyle.Quad, -- shape of the motion
    Enum.EasingDirection.Out -- slow down at the end
)

local tween = TweenService:Create(frame, info, {
    Position = UDim2.fromScale(0.5, 0.5),
    BackgroundTransparency = 0,
})
tween:Play()
```

Juice for the coin counter. Make it "pop" whenever the number changes:

```

local function pop(label: TextLabel)
    local uiScale = label:FindFirstChildOfClass("UIScale") or Instance.new("UIScale",
label)
    uiScale.Scale = 1.25
    TweenService:Create(uiScale, TweenInfo.new(0.25, Enum.EasingStyle.Back), { Scale = 1
}):Play()
end

```

Easing styles change the feeling: `Quad` is calm, `Back` overshoots playfully, `Bounce` is cartoony, `Linear` feels mechanical. Keep UI animations short (0.15–0.4 s). Slow UI feels laggy.

4.5 Main menu and HUD

Main menu. A full-screen frame with the game title, a big **PLAY** button, and a **SHOP** button (wired up in Module 7). Put a LocalScript in the menu ScreenGui:

```

local TweenService = game:GetService("TweenService")

local menu = script.Parent:WaitForChild("Menu")
local playButton = menu:WaitForChild("PlayButton")

playButton.Activated:Connect(function()
    local tween = TweenService:Create(menu, TweenInfo.new(0.3), { Position =
UDim2.fromScale(0, -1) })
    tween:Play()
    tween.Completed:Wait()
    menu.Visible = false
end)

```

Use `Activated` instead of `MouseButton1Click`. It works for mouse, touch and gamepad.

HUD coin counter. This LocalScript in `HUD` shows the player's coins. The `leaderstats` it reads are created in Module 6; until then it simply shows 0.

```

local Players = game:GetService("Players")
local TweenService = game:GetService("TweenService")

local player = Players.LocalPlayer
local coinText = script.Parent:WaitForChild("CoinPanel"):WaitForChild("CoinText")

local function pop()
    local s = coinText:FindFirstChildOfClass("UIScale") or Instance.new("UIScale",
coinText)
    s.Scale = 1.25
    TweenService:Create(s, TweenInfo.new(0.25, Enum.EasingStyle.Back), { Scale = 1
}):Play()
end

local leaderstats = player:WaitForChild("leaderstats")
local coins = leaderstats:WaitForChild("Coins")

coinText.Text = tostring(coins.Value)
coins.Changed:Connect(function(value)
    coinText.Text = tostring(value)
    pop()
end)

```

Round timer. In Module 7 the server will store the round state in attributes on `ReplicatedStorage`. The HUD just listens:

```

local ReplicatedStorage = game:GetService("ReplicatedStorage")
local timerText = script.Parent:WaitForChild("TimerPanel"):WaitForChild("TimerText")

local function refresh()
    local status = ReplicatedStorage:GetAttribute("RoundStatus") or "Waiting"
    local timeLeft = ReplicatedStorage:GetAttribute("TimeLeft") or 0
    timerText.Text = if timeLeft > 0 then `${status} · {timeLeft}s` else status
end

ReplicatedStorage:GetAttributeChangedSignal("RoundStatus"):Connect(refresh)
ReplicatedStorage:GetAttributeChangedSignal("TimeLeft"):Connect(refresh)
refresh()

```

TIP The backtick string `\${status} · {timeLeft}s` is Luau string interpolation, a clean way to build text from values.

TRY IT

1. Add a subtle idle animation to the PLAY button (scale 1 → 1.05 and back, repeating). Hint: `TweenInfo.new(0.8, Enum.EasingStyle.Sine, Enum.EasingDirection.InOut, -1, true)`.
2. Test the menu and HUD in the device emulator at phone size.

Module recap

- ☐ Intro screen runs once on join
- ☐ HUD with coin counter and round timer, built with Scale and AnchorPoint
- ☐ Main menu with PLAY that animates away
- ☐ Everything checked in the device emulator